

#2. Classical ML for Tabular Data

Instructor: **Hankook Lee** @ Efficient Learning Lab., Sungkyunkwan University

Lecture Overview

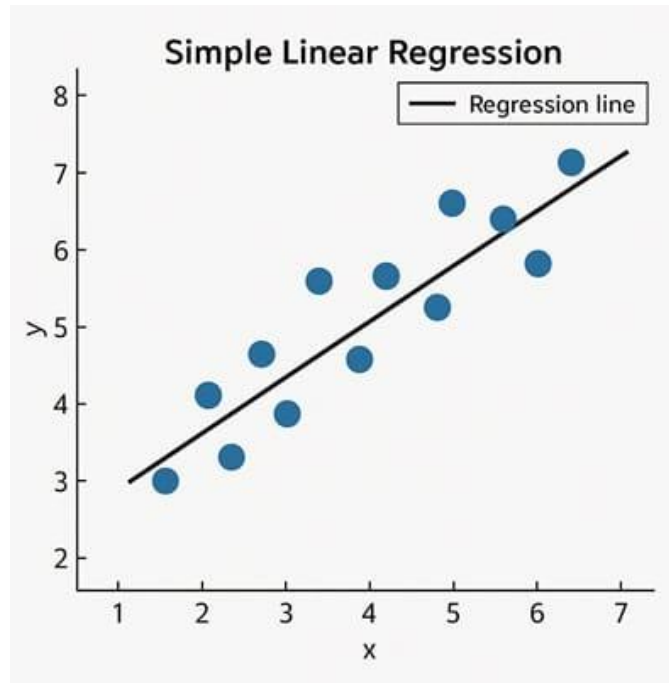
- This lecture covers **classical ML methods**:
 - Simple baselines – linear regression, logistic regression, k-nearest neighbors
 - Decision trees
 - Ensembles – bagging vs boosting, random forest, isolation forest
 - Gradient boosting and modern libraries – XGBoost, LightGBM, CatBoost
- **Why classical ML methods?**
 - These methods remain **strong baselines for tabular data**
 - Simplicity, strong performance, interpretability, and fast engineering
 - Tree-based approaches naturally handle heterogeneous features and remain dominant off-the-shelf methods for tabular prediction
 - Their core ideas are also widely used with modern approaches

Linear Regression

- Given features $\mathbf{x} \in \mathbb{R}^d$, a **linear regression** model predicts:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

- $\mathbf{w} \in \mathbb{R}^d$ is a vector of **weights**, $b \in \mathbb{R}$ is a scalar **bias**
- Geometrically, the model fits a hyperplane through the data



Linear Regression

- Modeling by minimizing the **mean squared error (MSE)**:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

- A training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ is given
- A closed-form solution exists: $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
 - Computational cost: $O(nd^2 + d^3) \rightarrow$ tractable for typical tabular sizes
 - When it is not tractable, one can find the best model via gradient-based optimization
- This corresponds to **maximum likelihood estimation** under Gaussian noise

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\mathbf{w}^T \mathbf{x} + b, \sigma^2)$$

Linear Regression

- **Overfitting** when d is large or features are correlated
- With **regularization** → reduce overfitting

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2 + \lambda \Omega(\mathbf{w})$$

- Ridge (L2) regression: $\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2 \rightarrow$ Shrinks weights smoothly toward zero
- Lasso (L1) regression: $\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 \rightarrow$ Drives some weights exactly to zero
- The hyperparameter λ can be chosen by cross-validation

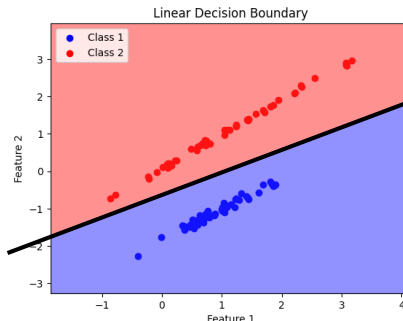
Logistic Regression

- Consider **binary classification** ($y \in \{0,1\}$)
 - We want to predict the **probability** $P(y = 1 | x) \in [0,1]$
 - But, a raw linear score $\mathbf{w}^T \mathbf{x} + b$ can be anywhere in $(-\infty, \infty)$
 - **Solution:** squash the raw score into a probability with the logistic sigmoid

- A **logistic regression** model predicts the probability

$$P(y = 1 | x) = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad \sigma(z) = 1/(1 + e^{-z})$$

- **Decision boundary:** $P(y = 1 | x) = P(y = 0 | x) \Leftrightarrow \mathbf{w}^T \mathbf{x} + b = 0$
 - $\mathbf{w}^T \mathbf{x} + b > 0 \rightarrow \hat{y} = 1$
 - $\mathbf{w}^T \mathbf{x} + b < 0 \rightarrow \hat{y} = 0$



Linear decision boundary

Logistic Regression

- Loss: **Cross-entropy**

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_i [y_i \log P(y = 1 \mid \mathbf{x}_i) + (1 - y_i) \log P(y = 0 \mid \mathbf{x}_i)]$$

- It is equivalent to the **negative log-likelihood** under a Bernoulli model
- It is convex, so optimization is reliable

- **No closed-form solution** exists $\rightarrow$ minimize the loss iteratively
 - E.g., gradient descent

- For multiclass classification, replace sigmoid with **softmax**:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_j \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}$$

- For regularization, apply L1 or L2 penalties as in linear regression

Linear/Logistic Regression

- **Strengths**

- Simple, fast, and highly interpretable
- Strong baselines for regression and classification

- **Limitations**

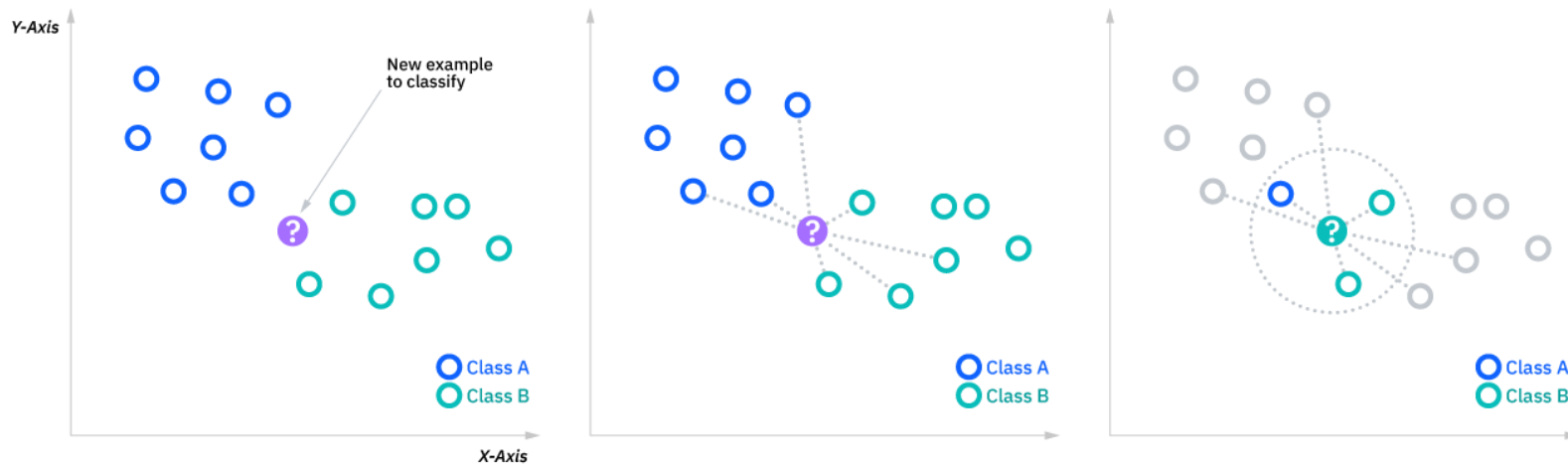
- Cannot directly capture non-linear patterns or complex feature interactions
- May be affected by outliers or extreme feature values
- Often require preprocessing and feature engineering

- **Takeaway:** Linear models are simple but powerful baselines

- Even when linear assumptions do not hold exactly, linear models remain useful practical baselines

k-Nearest Neighbors (kNN)

- **Key Idea:** "You are similar to your k nearest neighbors"
 - To predict for a new point $\mathbf{x}$, find the k training points closest to $\mathbf{x}$
 - **Classification:** return the majority vote
 - **Regression:** return the mean (or weighted mean) of their targets



- kNN has **no training step** → the dataset itself is the model

k-Nearest Neighbors (kNN)

- The notion of "closest" depends on a **distance metric**:
 - The default choice is **Euclidean distance**:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_j (x_j - x'_j)^2}$$

- You can use other distance metrics (e.g., Manhattan or Cosine distances)
- **Feature scaling** is critical
 - Features with large numerical ranges can dominate the distance
 - E.g., "Annual Income" may overwhelm "Age" unless scaled
- **Feature representation** also matters
 - Irrelevant features can distort neighborhoods
 - Categorical and missing values require careful preprocessing

k-Nearest Neighbors (kNN)

- Choosing the number of neighbors k
 - **Small k** (e.g., $k = 1$) → Highly local and flexible prediction
 - Sensitive to noise (i.e., overfitting)
 - **Large k** → Smoother and more stable prediction
 - Ignore local patterns
- How to choose?
 - Treat k as a hyperparameter → select the best one using cross-validation
 - For classification, use an odd k can reduce ties
- **Weighted kNN** is useful when k is large

$$\hat{y} = \frac{1}{\sum_i w_i} \sum_i w_i y_i, \quad w_i = 1/d(\mathbf{x}, \mathbf{x}_i)$$

k-Nearest Neighbors (kNN)

- **Strengths**

- Simple and intuitive
- Can capture non-linear decision boundaries

- **Limitations**

- Prediction can be slow and memory-intensive
- Sensitive to feature scaling and irrelevant features
- Performs poorly in high-dimensional spaces

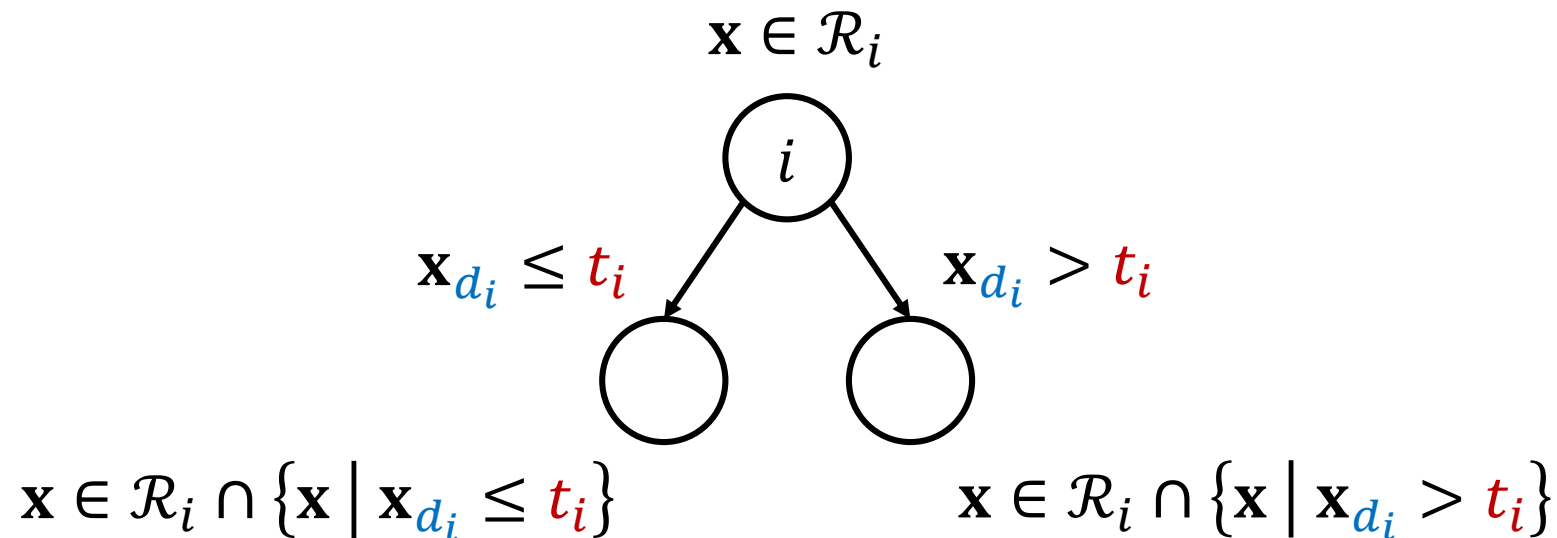
- **Takeaway:** kNN is a useful baseline, but its performance strongly depends on data representation and dimensionality

Decision Trees

- **Decision trees** are defined by (1) **recursively partitioning** the input space, and (2) defining a **local model in each region** of input space

Decision Trees

- **Decision trees** are defined by (1) **recursively partitioning** the input space, and (2) defining a **local model in each region** of input space
 - At each **internal node** i ,
 - the **feature dimension** d_i of the input vector $\mathbf{x}$ is compared to a **threshold value** t_i ,
 - the input is then passed down to the left or right branch



Decision Trees

- **Decision trees** are defined by (1) **recursively partitioning** the input space, and (2) defining a **local model in each region** of input space
 - At each **leaf node** j ,
 - the model **predicts the value** $\hat{y}_j$ for **any input that falls into the leaf node**

- Formally, a regression tree can be defined by

$$f(\mathbf{x}; \boldsymbol{\theta}) = \sum_j \hat{y}_j \mathbb{I}(\mathbf{x} \in \mathcal{R}_j)$$

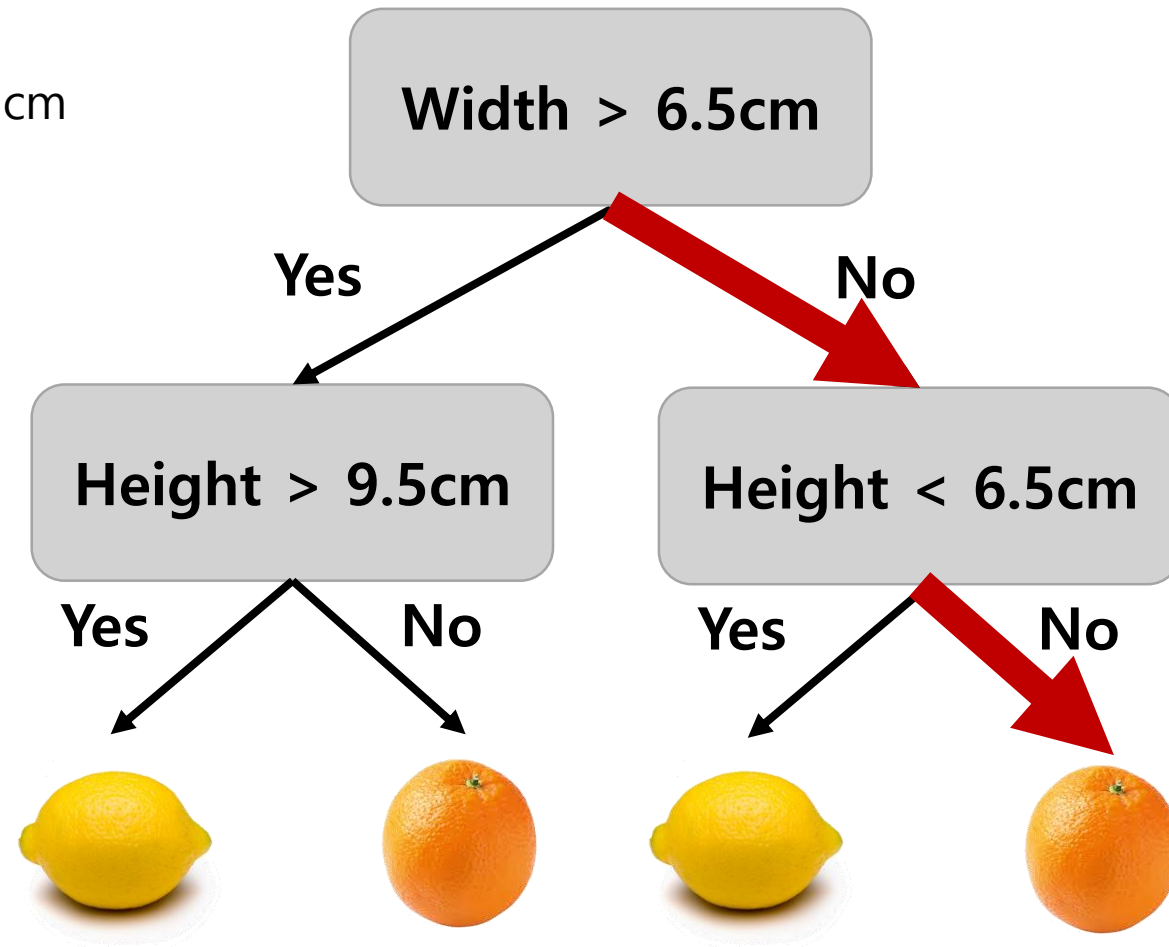
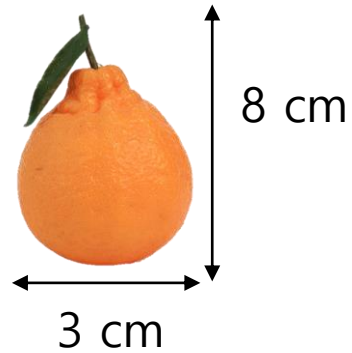
- Given a dataset $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}$, the prediction value can be defined as

$$\hat{y}_j = (\sum_n y_n \mathbb{I}(\mathbf{x}^{(n)} \in \mathcal{R}_j)) / (\sum_n \mathbb{I}(\mathbf{x}^{(n)} \in \mathcal{R}_j))$$

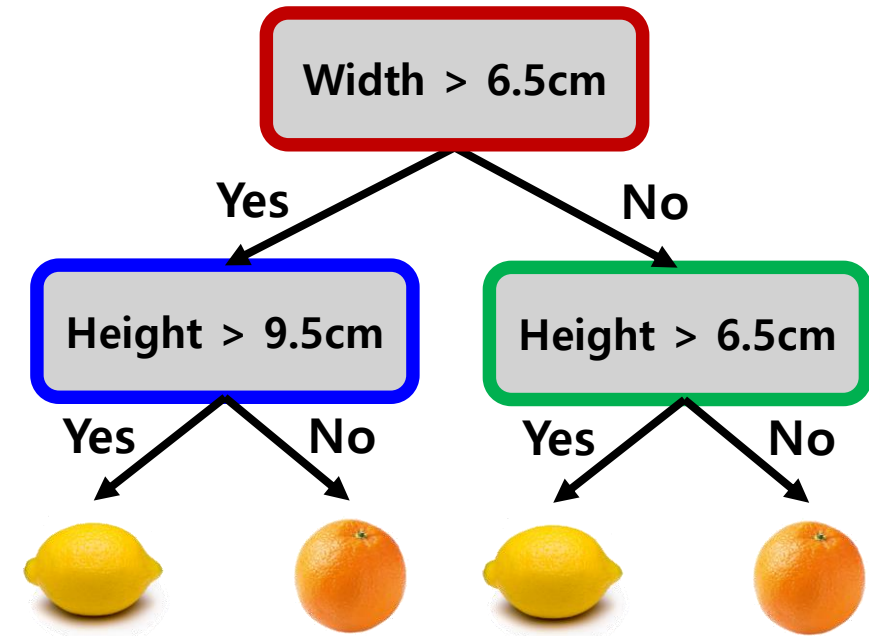
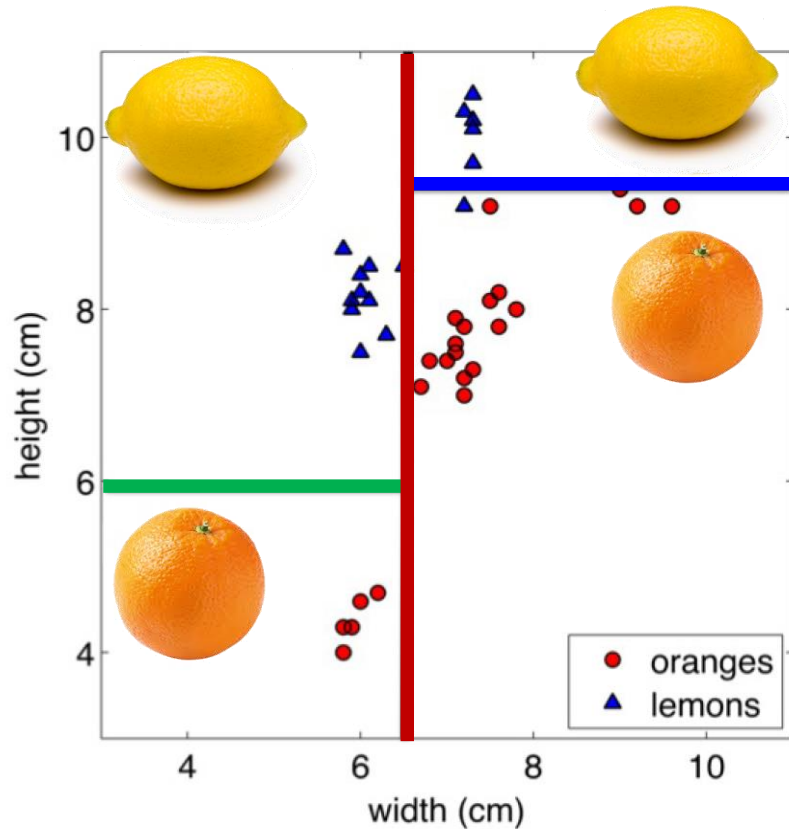
- $\hat{y}_j$ is the mean target or the majority class

Decision Trees: An Example

Test sample



Decision Trees: An Example



Decision Trees: Model Fitting

- To fit the model, we need to minimize the following loss:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{|\mathcal{D}|} \sum_n \ell \left(y^{(n)}, f(\mathbf{x}^{(n)}; \boldsymbol{\theta}) \right) = \frac{1}{|\mathcal{D}|} \sum_j \sum_{\mathbf{x}^{(n)} \in \mathcal{R}_j} \ell \left(y^{(n)}, f(\mathbf{x}^{(n)}; \boldsymbol{\theta}) \right)$$

- **(Q)** Can we find an optimal solution?
- **(A)** Unfortunately, we cannot
 - This objective is **not differentiable**
 - Finding the optimal partitioning of the data is **NP-complete**
- Alternatively, use a **greedy** procedure!
 - We **iteratively grow the tree one node** at each iteration

Decision Trees: Model Fitting

- First, reformulate the loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{|\mathcal{D}|} \sum_n \ell \left(y^{(n)}, f(\mathbf{x}^{(n)}; \boldsymbol{\theta}) \right) = \sum_{j \in \mathcal{J}} \frac{|\mathcal{D}_j|}{|\mathcal{D}|} \sum_{\mathbf{x}^{(n)} \in \mathcal{R}_j} \frac{1}{|\mathcal{D}_j|} \ell \left(y^{(n)}, f(\mathbf{x}^{(n)}; \boldsymbol{\theta}) \right)$$

Decision Trees: Model Fitting

- First, reformulate the loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{|\mathcal{D}|} \sum_n \ell(y^{(n)}, f(\mathbf{x}^{(n)}; \boldsymbol{\theta})) = \sum_{j \in \mathcal{J}} \frac{|\mathcal{D}_j|}{|\mathcal{D}|} \sum_{\mathbf{x}^{(n)} \in \mathcal{R}_j} \frac{1}{|\mathcal{D}_j|} \ell(y^{(n)}, f(\mathbf{x}^{(n)}; \boldsymbol{\theta}))$$

the set of leaf nodes

the expected loss at the node j

the probability of reaching the node j

Decision Trees: Model Fitting

- First, reformulate the loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{j \in \mathcal{J}} p(j) \ell(j)$$

- $\mathcal{J}$ is the set of leaf nodes
 - $p(j)$ is the probability of reaching the leaf node $j \in \mathcal{J}$
 - $\ell(j)$ is the expected loss function at the leaf node $j \in \mathcal{J}$
- Formally, we can write $p(\cdot)$ and $\ell(\cdot)$ as:
 - $p(j) = P(\mathbf{x} \in \mathcal{R}_j)$
 - $\ell(j) = \sum_{(\mathbf{x}, y)} P(\mathbf{x}, y | \mathbf{x} \in \mathcal{R}_j) \ell(y, \hat{y}_j)$

Decision Trees: Model Fitting

- First, reformulate the loss function:

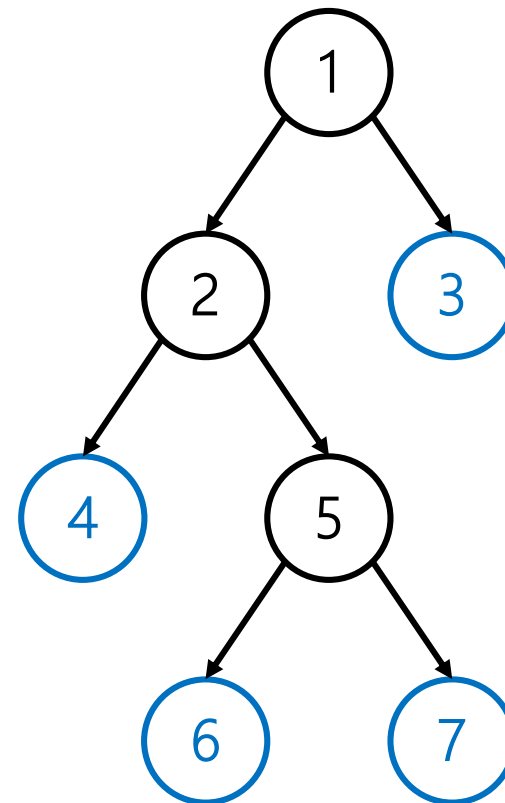
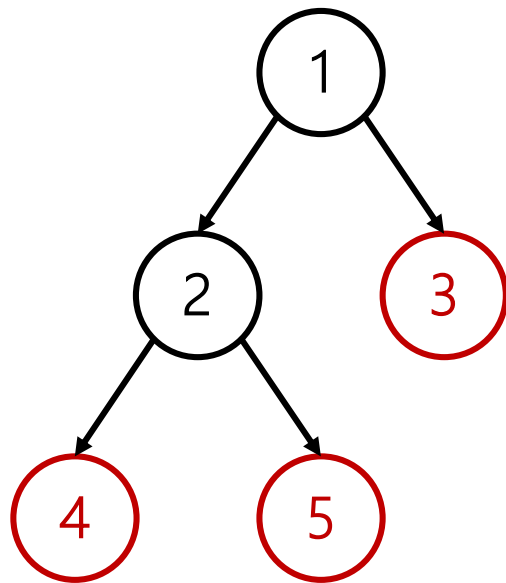
$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{j \in \mathcal{J}} p(j) \ell(j)$$

- $\mathcal{J}$ is the set of leaf nodes
 - $p(j)$ is the probability of reaching the leaf node $j \in \mathcal{J}$
 - $\ell(j)$ is the expected loss function at the leaf node $j \in \mathcal{J}$
-
- Formally, we can write $p(\cdot)$ and $\ell(\cdot)$ as:
 - $p(j) = P(\mathbf{x} \in \mathcal{R}_j) = |\mathcal{D}_j|/|\mathcal{D}|$
 - $\ell(j) = \sum_{(\mathbf{x}, y)} P(\mathbf{x}, y | \mathbf{x} \in \mathcal{R}_j) \ell(y, \hat{y}_j) = \sum_{(\mathbf{x}, y) \in \mathcal{D}_j} \ell(y, \hat{y}_j) / |\mathcal{D}_j|$
 - $\mathcal{D}$ is the set of all training samples
 - $\mathcal{D}_j = \{(\mathbf{x}, y) \in \mathcal{D} : \mathbf{x} \in \mathcal{R}_j\}$ is the set of samples in the region $\mathcal{R}_j$

Decision Trees: Model Fitting

- When splitting a leaf node k to $\{k_L, k_R\}$, the objective changes as:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{j \in \mathcal{J}} p(j) \ell(j) \rightarrow \mathcal{L}(\boldsymbol{\theta}') = \sum_{j \in \mathcal{J} \setminus \{k\} \cup \{k_L, k_R\}} p(j) \ell(j)$$



$$\mathcal{J}' = \{3, 4, 6, 7\}$$

Decision Trees: Model Fitting

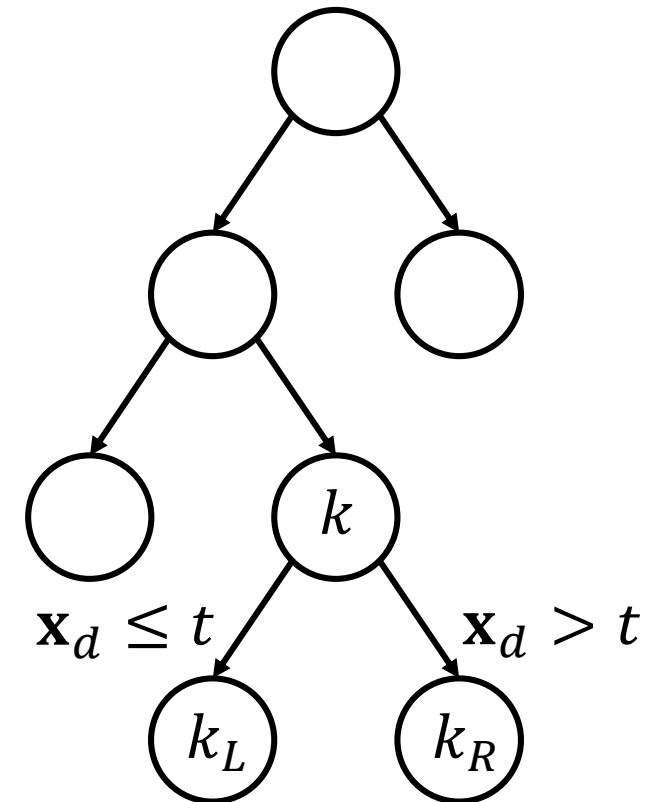
- When splitting a leaf node k to $\{k_L, k_R\}$, the objective changes as:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{j \in \mathcal{J}} p(j) \ell(j) \rightarrow \mathcal{L}(\boldsymbol{\theta}') = \sum_{j \in \mathcal{J} \setminus \{k\} \cup \{k_L, k_R\}} p(j) \ell(j)$$

$$\bullet \mathcal{L}(\boldsymbol{\theta}') = \mathcal{L}(\boldsymbol{\theta}) - p(k) \underbrace{\left(\ell(k) - \frac{p(k_L)}{p(k)} \ell(k_L) - \frac{p(k_R)}{p(k)} \ell(k_R) \right)}_{\Delta(k, d, t)}$$

- A split can be defined by a leaf node k , a feature dimension d , and a threshold value t
- Find the best split (k, d, t) at each iteration!

$$\arg \max_{k \in \mathcal{J}, d, t} p(k) \Delta(k, d, t)$$

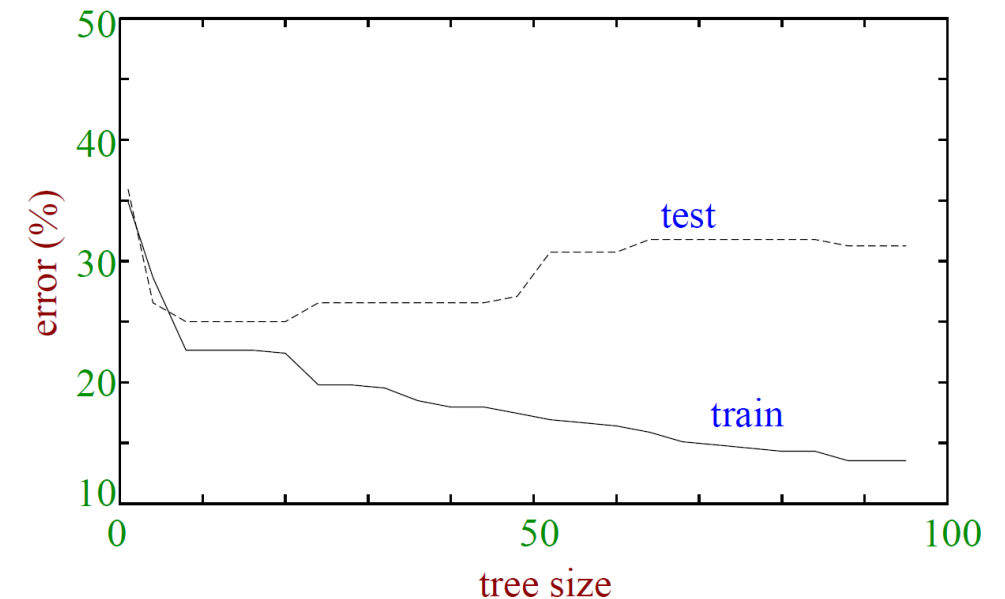
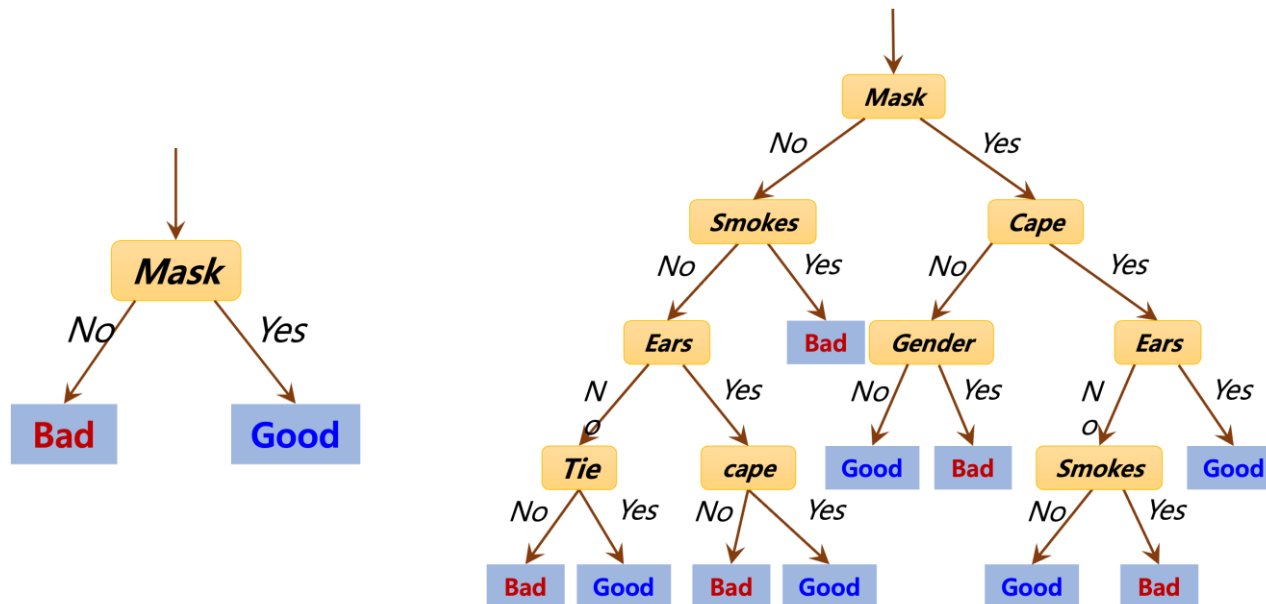


Decision Trees: Model Fitting

- In the greedy algorithm,
 - The loss is monotonically decreasing
 - The split improvement $\Delta(k, d, t)$ can be re-used until splitting k
- **(Q)** How to design the loss function for each node?
- **(A)** Assign a single prediction $\hat{y}_k$ for each node k and compute error
 - For regression, $\hat{y}_k = \frac{1}{|\mathcal{D}_k|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_k} y$ and $\ell(k) = \frac{1}{|\mathcal{D}_k|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_k} (y - \hat{y}_k)^2$
 - For classification, $\hat{y}_{kc} = \frac{1}{|\mathcal{D}_k|} \sum_{(\mathbf{x}, y) \in \mathcal{D}_k} \mathbb{I}(y = c)$ and $\ell(k) = \sum_c \hat{y}_{kc} (1 - \hat{y}_{kc})$
 - This is called Gini index
 - For classification, you can use the entropy $\ell(k) = -\sum_c \hat{y}_{kc} \log \hat{y}_{kc}$

Decision Trees: Underfitting & Overfitting

- **Tree complexity** controls **underfitting** vs **overfitting**
 - **Too shallow** → miss important patterns → underfitting
 - **Too deep** → memorize noise in training data → overfitting
 - A fully grown tree can achieve nearly perfect training accuracy, but generalize poorly



Decision Trees: Underfitting & Overfitting

- **Tree complexity** controls **underfitting** vs **overfitting**
 - **Too shallow** → miss important patterns → underfitting
 - **Too deep** → memorize noise in training data → overfitting
 - A fully grown tree can achieve nearly perfect training accuracy, but generalize poorly
- How to control tree complexity?
 - **Pre-pruning:** Stop growing the tree before it becomes too complex
 - E.g., `max_depth`, `min_samples_leaf`, `min_impurity_decrease`
 - **Post-pruning:** Grow a large tree first, then remove unnecessary branches
 - Cost-complexity pruning balances predictive fit and tree size

Decision Trees

- **Strengths**

- Rule-based predictions that are easy to understand
- Captures non-linear patterns and feature interactions
- Requires little preprocessing; no feature scaling required

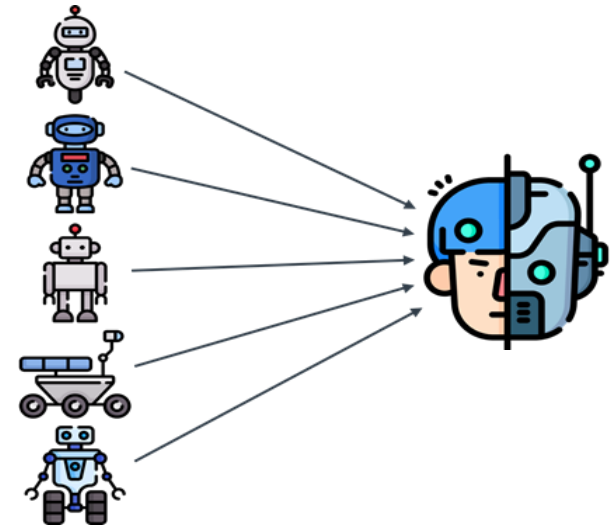
- **Limitations**

- Prone to overfitting when grown too deeply
- Unstable: small data changes can produce different trees
- May generalize poorly without complexity control

- **Decision trees are highly interpretable building blocks** for powerful ensemble methods

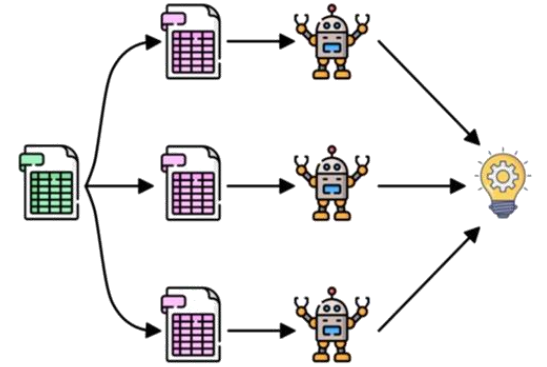
Ensemble Methods

- It utilizes the prediction of **multiple base models** to improve **generalizability** and **robustness** over a single model
- They must differ somehow
 - Trained with different data
 - Different algorithms
 - Different choices of hyperparameters
- It is usually easy to implement
 - The hard part is **how to design ensembles** according to the goal

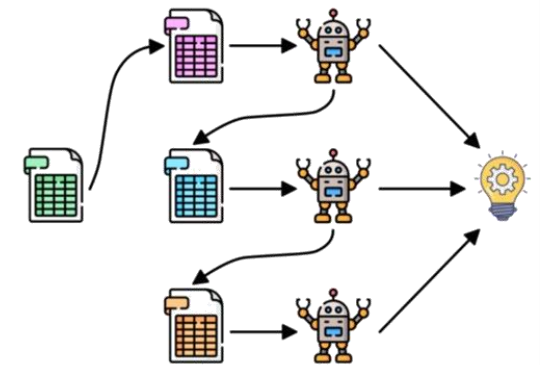


Types of Ensemble Methods

- Bagging
 - It is the short form for **bootstrap aggregating**
 - Train classifiers **independently** on random subsets of training data
 - E.g., Random Forest
- Boosting
 - Train classifiers **sequentially**
 - Learns **from previous predictor mistakes** to make better predictions
 - E.g., AdaBoost



Parallel



Sequential

Motivation of Bagging

- We sample m independent training sets $\{\mathcal{D}_i\}_{i=1}^m$
- Learn a classifier h_i for each $\mathcal{D}_i$ and take the average for prediction:

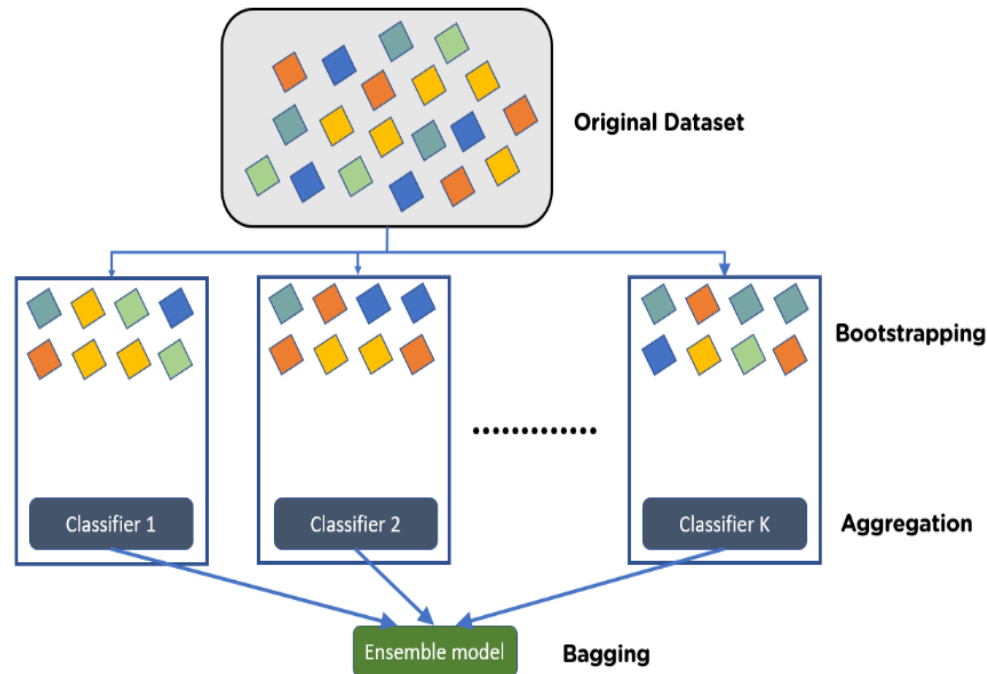
$$\hat{y} = \frac{1}{m} \sum_{i=1}^m \hat{y}_i \quad \text{where} \quad \hat{y}_i = h_i(\mathbf{x})$$

- **(Q)** How does it affect **bias** and **variance** of the expected loss?
- **(A)** Bias is unchanged, but variance is reduced

$$\begin{aligned} \mathbb{E}[\hat{y}] &= \mathbb{E}\left[\frac{1}{m} \sum_{i=1}^m \hat{y}_i\right] = \frac{1}{m} \mathbb{E}\left[\sum_{i=1}^m \hat{y}_i\right] = \mathbb{E}[\hat{y}_i] \\ \text{Var}[\hat{y}] &= \text{Var}\left[\frac{1}{m} \sum_{i=1}^m \hat{y}_i\right] = \frac{1}{m^2} \sum_{i=1}^m \text{Var}[\hat{y}_i] = \frac{1}{m} \text{Var}[\hat{y}_i] \end{aligned}$$

Bagging

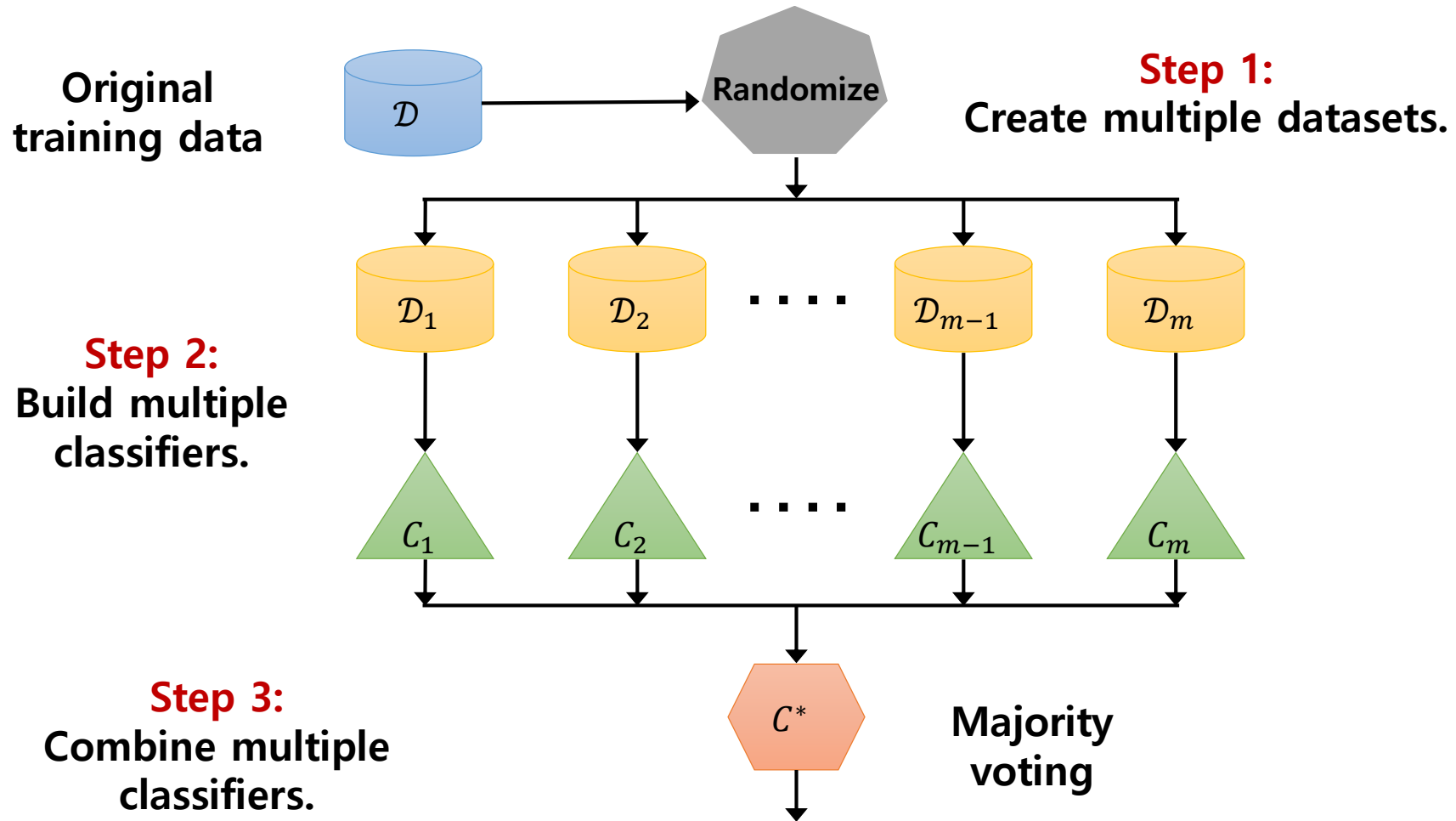
- **Bootstrap Aggregating** (Bagging): construct multiple datasets from $\mathcal{D}$
 - $\mathcal{D}$ is the training set of n samples
 - Generate m datasets by sampling n training samples **with replacement**
 - Average the predictions of models independently trained on each dataset



Bagging

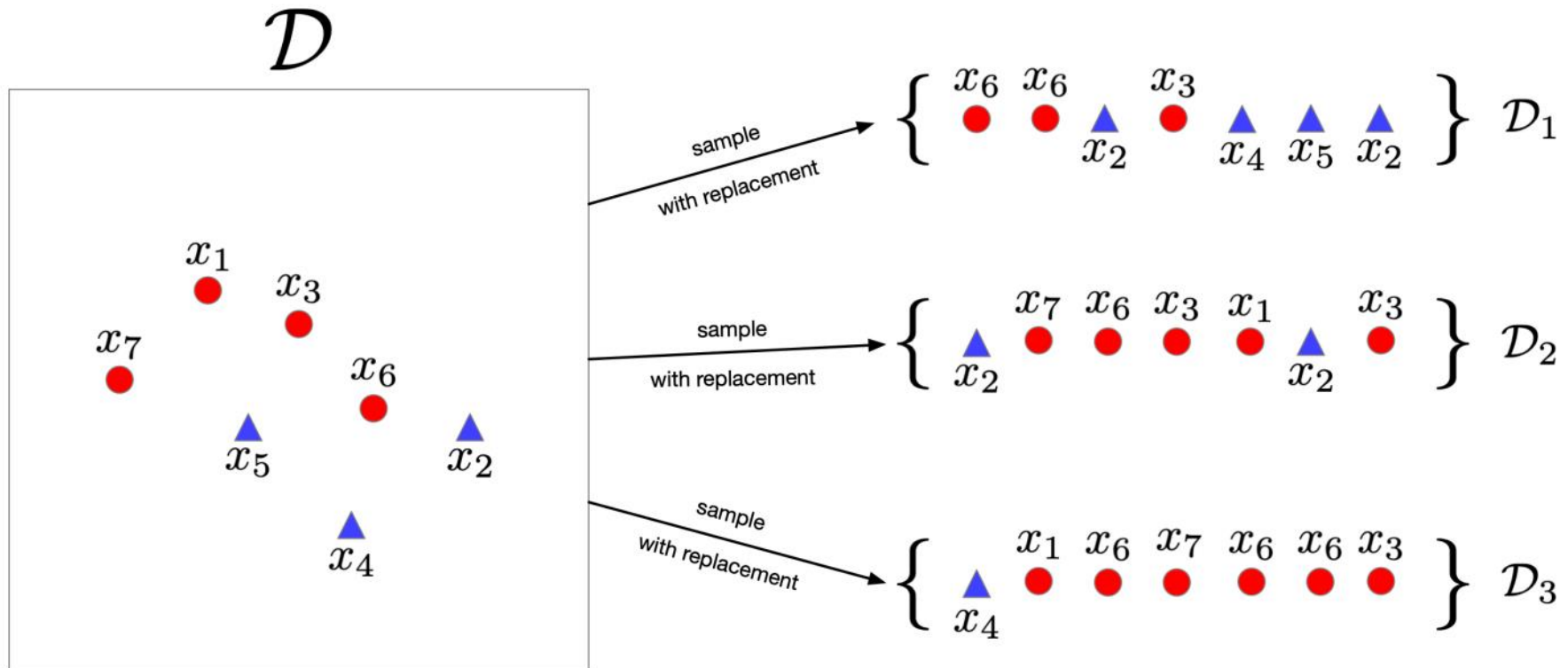
- **Bootstrap Aggregating** (Bagging): construct multiple datasets from $\mathcal{D}$
 - $\mathcal{D}$ is the training set of n samples
 - Generate m datasets by sampling n training samples **with replacement**
 - Average the predictions of models independently trained on each dataset
- Building a classifier on each dataset
 - Each sample has $(1 - 1/n)^n$ probability of being not selected
 - Each dataset is expected to have $(1 - 1/e) \approx 63\%$ unique training examples
 - You can use remaining samples as the validation set

Overall Process of Bagging



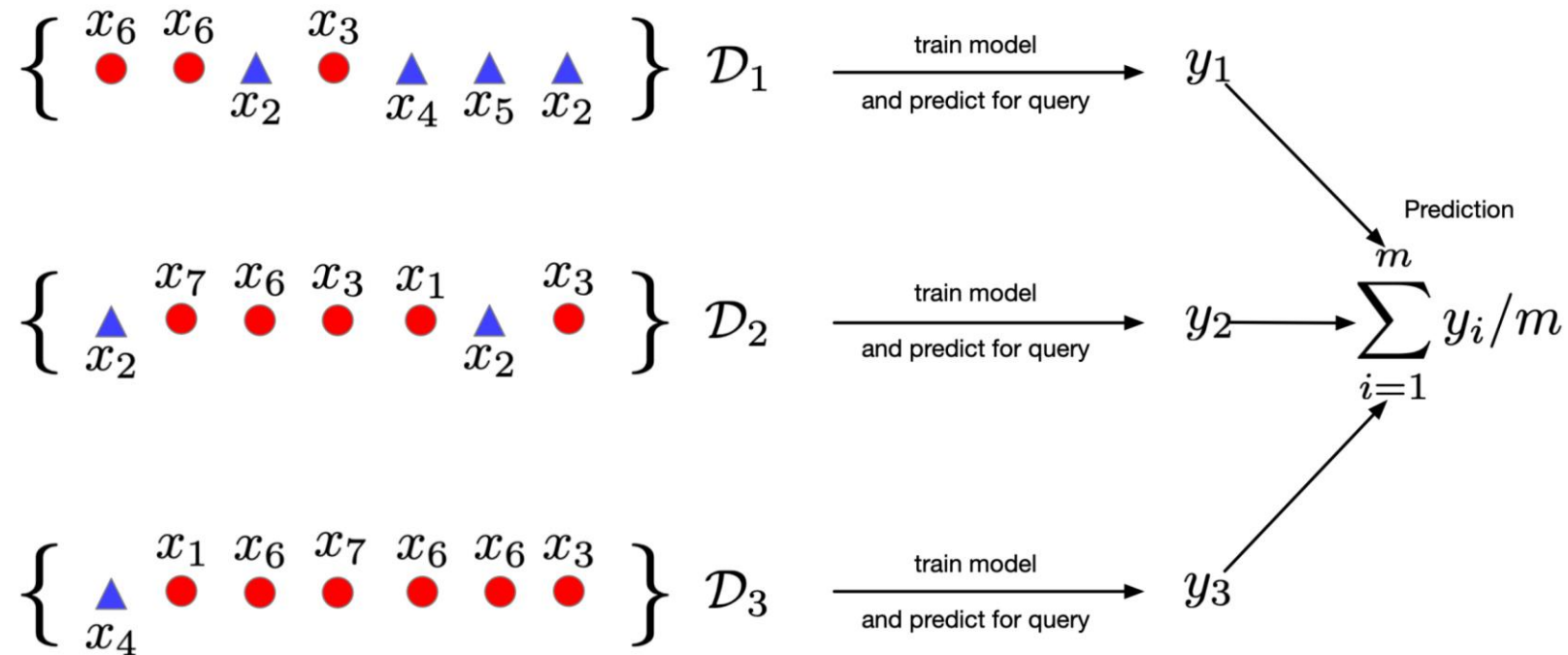
Bagging Example

- In this example, $n = 7, m = 3$



Bagging Example

- Predicting a new sample $\mathbf{x}$



Effect of Correlation

- **Problem:** The datasets are not perfectly independent, so, we cannot not get the $1/m$ variance reduction.

- If the sampled predictions have variance σ^2 and correlation ρ ,

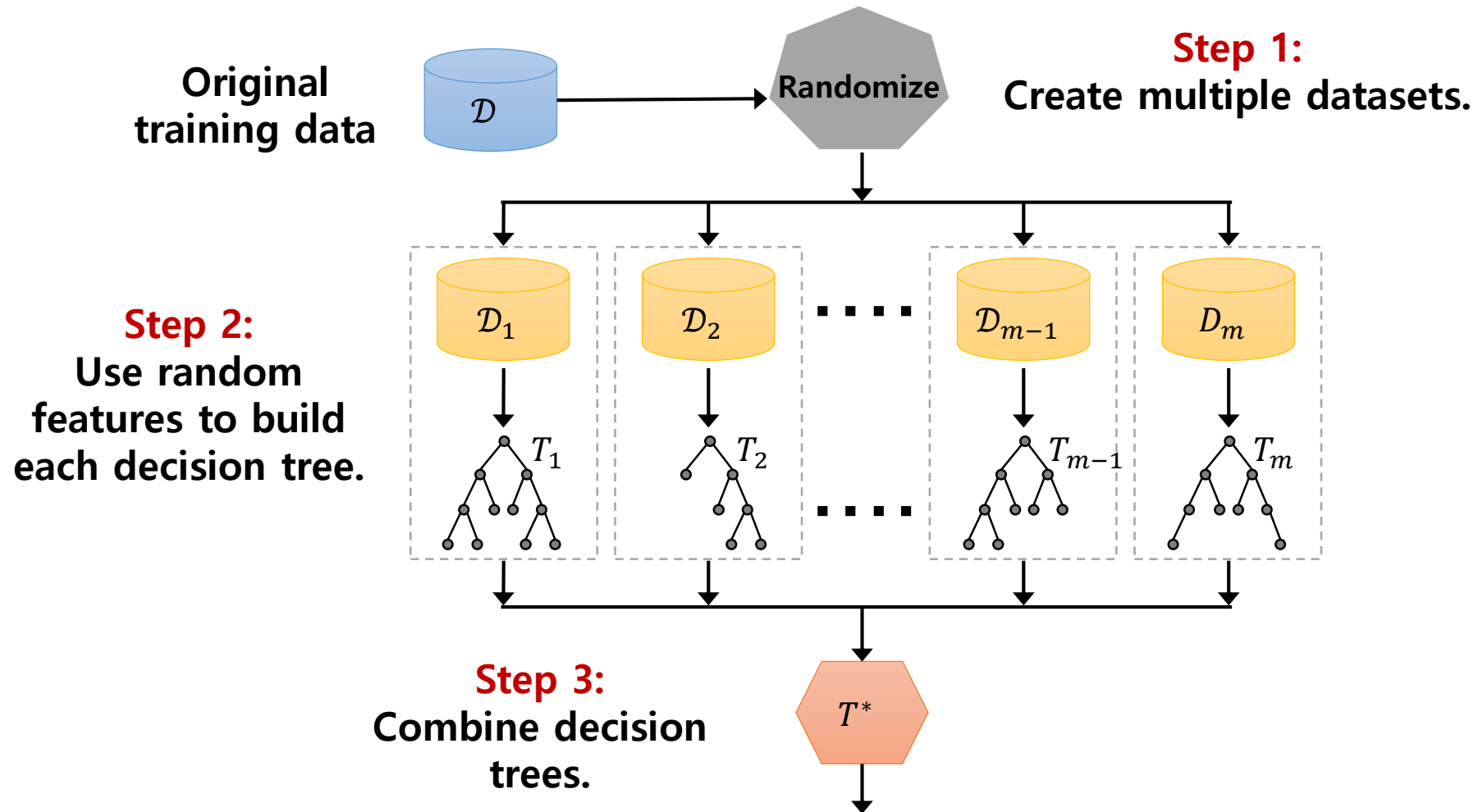
$$\text{Var}[\hat{y}] = \text{Var}\left[\frac{1}{m} \sum_{i=1}^m \hat{y}_i\right] = \frac{1}{m} (1 - \rho) \sigma^2 + \rho \sigma^2$$

- **Solution:** Introduce **additional variability** into the model to reduce sample correlation
 - It is helpful to use multiple configurations of the same model

Random Forests

- Random Forest (RF) is a **bagged decision tree** with an extra trick to decorrelate the predictions
- Utilize two sources of randomness: **Bagging** and **random feature vectors**
 - **Random dataset**: Each tree is grown using bootstrap sampling
 - **Random feature vectors**: **At each node, the best split is chosen from a random subset of attributes** instead of all the attributes
- Despite the simplicity, it often works well
 - It is one of the widely used models in Kaggle competitions

Random Forests



Random Forests

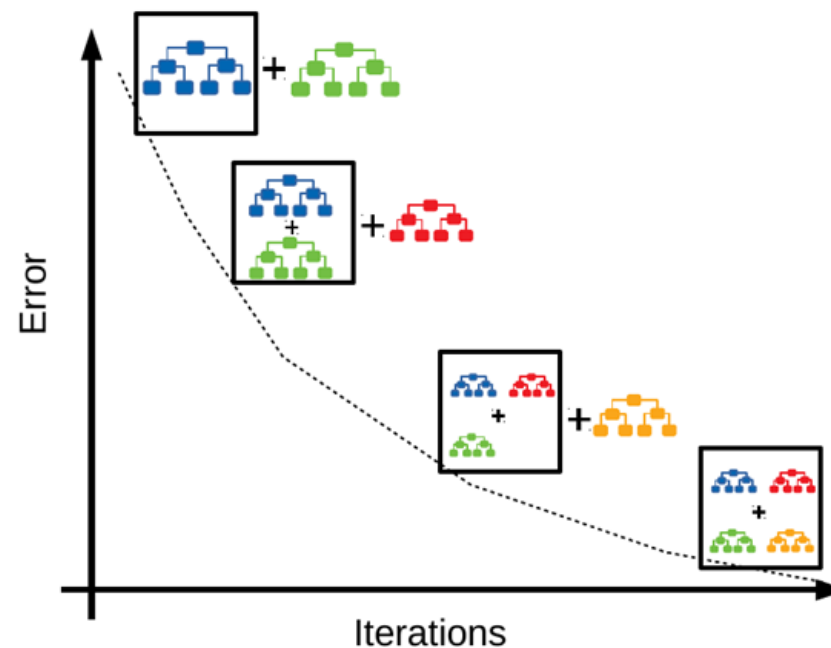
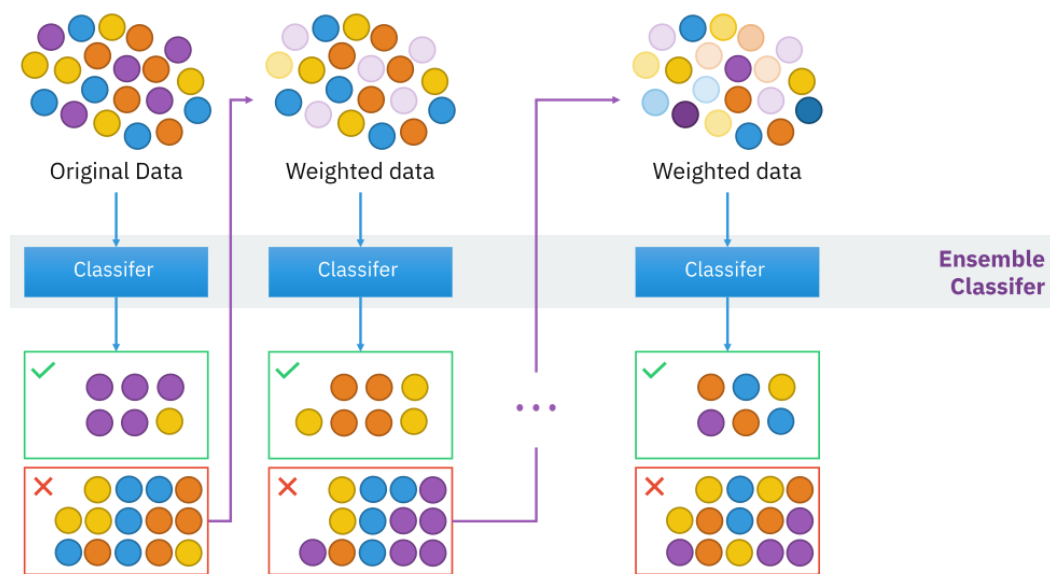
Algorithm 1 Random Forest

Precondition: A training set $S := (x_1, y_1), \dots, (x_n, y_n)$, features F , and number of trees in forest B .

```
1 function RANDOMFOREST( $S, F$ )
2    $H \leftarrow \emptyset$ 
3   for  $i \in 1, \dots, B$  do
4      $S^{(i)} \leftarrow$  A bootstrap sample from  $S$ 
5      $h_i \leftarrow$  RANDOMIZEDTREELEARN( $S^{(i)}, F$ )
6      $H \leftarrow H \cup \{h_i\}$ 
7   end for
8   return  $H$ 
9 end function
10 function RANDOMIZEDTREELEARN( $S, F$ )
11   At each node:
12      $f \leftarrow$  very small subset of  $F$ 
13     Split on best feature in  $f$ 
14   return The learned tree
15 end function
```

Boosting

- Train classifiers **sequentially**, focusing on **correcting the mistakes of the current ensemble**



- **AdaBoost**: gives more weight to previously misclassified samples
- **Gradient Boosting**: fits new models to reduce the remaining loss
 - Modern tabular ML libraries are primarily based on gradient boosted decision trees

GBDT: Gradient Boosting Decision Trees

- **GBDT** builds decision trees sequentially to correct the remaining errors of the current ensemble
 1. **Initial prediction:** $F_0(\mathbf{x}) = \bar{y}$
 2. **Remaining errors** after the current ensemble: $r_i^{(m)} = y_i - F_{m-1}(\mathbf{x}_i)$
 3. **Fit a decision tree** $h_m(\mathbf{x})$ to the remaining errors: $\left\{(\mathbf{x}_i, r_i^{(m)})\right\}_{i=1}^n$
 4. **Build an ensemble:** $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x})$
- Why is it called "**gradient**" boosting?
 - The remaining error $r_i^{(m)}$ is the gradient of the squared error of $\mathbf{x}_i$
 - $h_m(\mathbf{x})$ learns to predict the gradient
 - η is a learning rate

GBDT: Gradient Boosting Decision Trees

- GBDT gradually builds a strong model from **many shallow trees**
 - **Why shallow decision trees?**
 - Capture non-linearities and feature interactions out of the box
 - Efficiently learn pseudo-residuals
 - Add natural piecewise-constant corrections to the ensemble
 - **Key Hyperparameters**
 - Number of trees → more expressive, but may overfit
 - Learning rate η → faster correction, but less conservative
 - Tree depth / number of leaves → capture more interactions, but may overfit
 - Subsampling → improve robustness
- **GBDT combines weak corrective trees into a powerful predictor, but requires careful tuning**

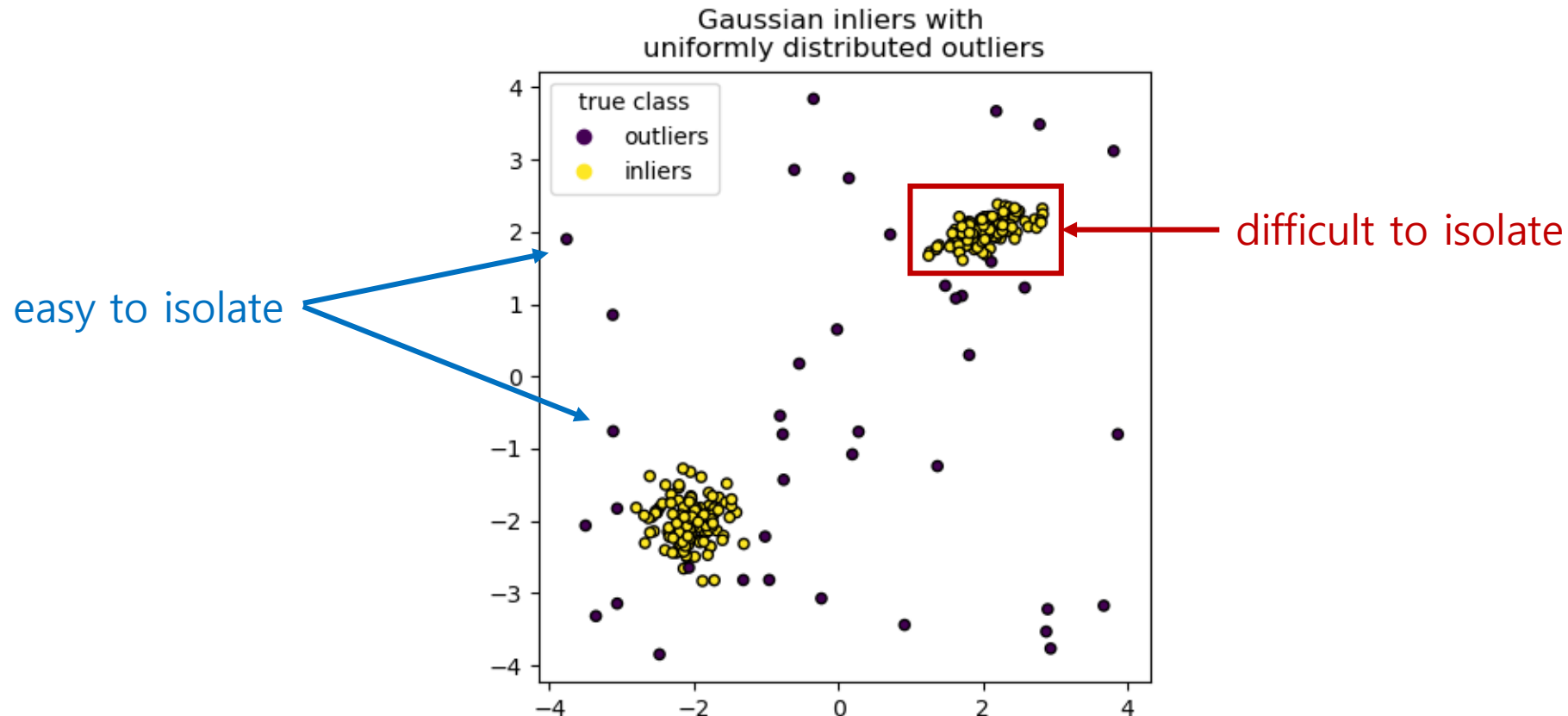
Modern GBDT Libraries

- GBDT has become widely used through highly optimized frameworks:
 - **XGBoost** (Chen & Guestrin, 2016)
 - **LightGBM** (Ke et al., 2017)
 - **CatBoost** (Prokhorenkova et al., 2018)
- All are **open-source** and **easy to use** in practice
- They share the GBDT principle, but address different practical challenges

Framework	Focus
XGBoost	Robust, scalable tree boosting
LightGBM	Fast and memory-efficient training
CatBoost	Effective handling of categorical features

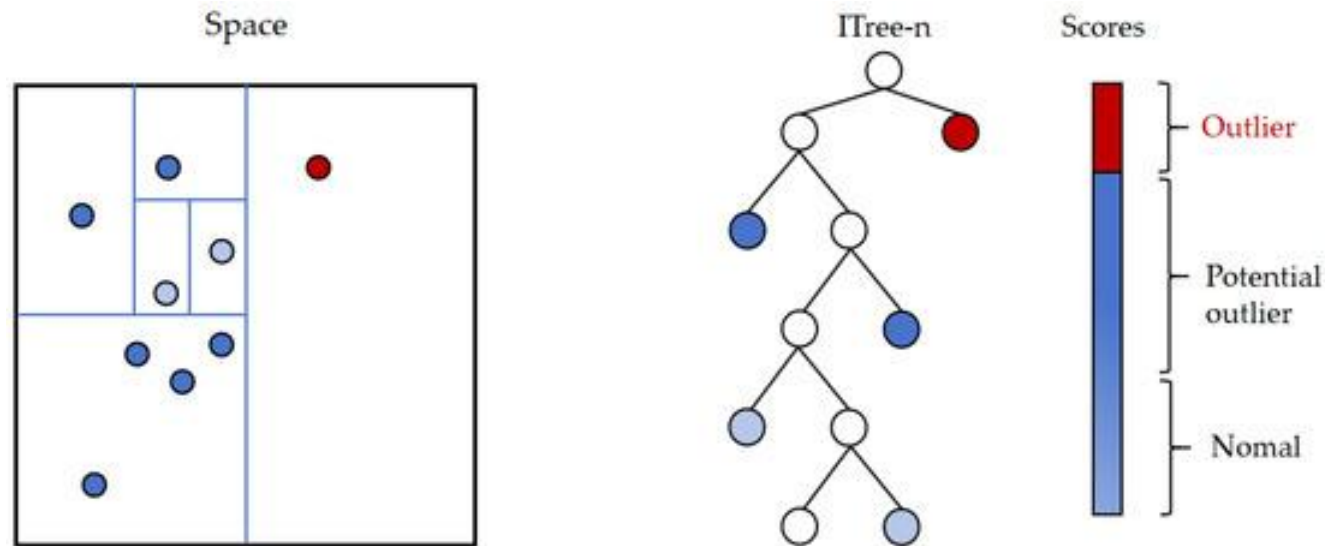
+ Isolation Forest for Anomaly Detection

- **Key Idea:** Anomalies are easier to isolate by random splits



+ Isolation Forest for Anomaly Detection

- **Key Idea:** Anomalies are easier to isolate by random splits
- **Building an Isolation Tree**
 - Randomly sample a subset of the training data
 - At each node, randomly choose a feature and a split value
 - Repeat until a sample is isolated or a maximum depth is reached



+ Isolation Forest for Anomaly Detection

- **Key Idea:** Anomalies are easier to isolate by random splits
- **Building an Isolation Tree**
 - Randomly sample a subset of the training data
 - At each node, randomly choose a feature and a split value
 - Repeat until a sample is isolated or a maximum depth is reached
- **From Isolation Tree to Isolation Forest**
 - Build many random isolation trees
 - Measure the average path length of each sample
 - Shorter average path length → Higher anomaly score
 - No class labels or anomaly labels are required

Summary

- **Classical ML remain strong and practical choices for tabular data**
 - Fast to train and easy to evaluate
 - Often highly competitive in predictive performance
 - Generally, more interpretable than complex models
- **Tree-based methods** are especially effective for tabular prediction
- For tabular data, simpler classical methods are not outdated alternatives; **they are often the first models to try**

Toy Exercise: ML with sklearn

- Download "MAGIC Gamma Telescope" dataset

```
# 1) 데이터 다운로드 - UCI 서버에서 단일 CSV 파일을 직접 받아옵니다.
DATA_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/magic/magic04.data"

NUM_COLS = [
    "fLength",    # 주축 길이
    "fWidth",     # 부축 너비
    "fSize",      # 픽셀 합의 log
    "fConc",      # 상위 2 픽셀 / fSize
    "fConc1",     # 최상위 픽셀 / fSize
    "fAsym",      # 주축 비대칭도
    "fM3Long",    # 주축 3차 모멘트
    "fM3Trans",   # 부축 3차 모멘트
    "fAlpha",     # 주축과 원점 벡터 사이 각도
    "fDist",      # 원점에서 중심까지 거리
]

TARGET = "class"
COLUMNS = NUM_COLS + [TARGET]

df = pd.read_csv(DATA_URL, header=None, names=COLUMNS)
print("shape:", df.shape)
df.head()
```

	fLength	fWidth	fSize	fConc	fConc1	fAsym	fM3Long	fM3Trans	fAlpha	fDist	class
0	28.7967	16.0021	2.6449	0.3918	0.1982	27.7004	22.0110	-8.2027	40.0920	81.8828	g
1	31.6036	11.7235	2.5185	0.5303	0.3773	26.2722	23.8238	-9.9574	6.3609	205.2610	g
2	162.0520	136.0310	4.0612	0.0374	0.0187	116.7410	-64.8580	-45.2160	76.9600	256.7880	g
3	23.8172	9.5728	2.3385	0.6147	0.3922	27.2107	-6.4633	-7.1513	10.4490	116.7370	g
4	75.1362	30.9205	3.1611	0.3168	0.1832	-5.5277	28.5525	21.8393	4.6480	356.4620	g

Toy Exercise: ML with sklearn

- Train-test split & preprocessing

```
# 2) 학습/테스트 분리 (8:2, 클래스 비율 유지)
from sklearn.model_selection import train_test_split

CAT_COLS = [] # MAGIC에는 범주형 feature가 없음 - 변수만 정의해 두기

# 라벨: 'g' (gamma, 신호) → 1, 'h' (hadron, 배경) → 0
df["y"] = (df[TARGET] == "g").astype(int)

train_df, test_df = train_test_split(
    df, test_size=0.2, stratify=df["y"], random_state=SEED,
)
print("train:", train_df.shape, "test:", test_df.shape)
print("\n== 타겟 분포 (train) ==")
print(train_df[TARGET].value_counts(normalize=True).round(3))
```

```
from sklearn.preprocessing import StandardScaler

def split_xy(d):
    return d[NUM_COLS].to_numpy(dtype=float), d["y"].to_numpy()

X_train_raw, y_train = split_xy(train_df)
X_test_raw, y_test = split_xy(test_df)

# (A) 트리 모델용: 원본 그대로
X_train_tree = X_train_raw
X_test_tree = X_test_raw

# (B) 선형/딥러닝 모델용: StandardScaler
scaler = StandardScaler().fit(X_train_raw)
X_train_lin = scaler.transform(X_train_raw)
X_test_lin = scaler.transform(X_test_raw)

print("Tree 형태 :", X_train_tree.shape, X_test_tree.shape)
print("Linear 형태:", X_train_lin.shape, X_test_lin.shape)
```

Toy Exercise: ML with sklearn

- ML models can be easily implemented using **sklearn** library

```
from sklearn.linear_model import LogisticRegression
```

```
lr_model = LogisticRegression(  
    max_iter=1000,  
    C=1.0,          # 규제 강도의 역수  
    solver="lbfgs",  
    random_state=SEED,  
)
```

```
lr_model.fit(X_train_lin, y_train)
```

```
y_pred_lr = lr_model.predict(X_test_lin)
```

```
y_score_lr = lr_model.predict_proba(X_test_lin)
```

```
evaluate("LogisticRegression", y_test, y_pred_lr, y_score_lr)
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt_model = DecisionTreeClassifier(  
    max_depth=8,          # 너무 깊으면 과적합  
    min_samples_leaf=20,  
    random_state=SEED,  
)
```

```
dt_model.fit(X_train_tree, y_train)
```

```
y_pred_dt = dt_model.predict(X_test_tree)
```

```
y_score_dt = dt_model.predict_proba(X_test_tree)[:, 1]
```

```
evaluate("DecisionTree", y_test, y_pred_dt, y_score_dt)
```

Toy Exercise: ML with sklearn

- Compare a variety of ML models

	Accuracy	F1	AUC
TabPFN (10000 train)	0.8938	0.9209	0.9452
XGBoost	0.8864	0.9148	0.9396
TabM	0.8707	0.9055	0.9274
DecisionTree	0.8496	0.8891	0.8866
LogisticRegression	0.7836	0.8439	0.8313
LLM (Qwen2.5-0.5B, 200 samples)	0.4200	0.0000	0.7283

- We will also cover **recent deep learning approaches** for tabular data, including **TabM**, **LLM-based models**, and **TabPFN**

Thank You for Your Attention!
